

Inigo Quilez :: articles :: hardware interpolation - 2013



Intro

Hardware texture interpolation of 8 bit textures (RGBA8 and variants) is fast and convenient. It is bilinear (plus mipmapping), and despite **it can be somehow improved**, it works great for most cases. Most cases being texture mapping of surfaces with color/albedo, normal and specular textures for low dynamic range or softly lit scenes. Generally these types of textures don't need lots of color fidelity (where "color" means pixel value/content), and therefore the hardware's texture interpolation units implement some simple interpolation based on 24.8 fixed point arithmetic. That's what all GPUs' hardware implements these days.

A 24.8 precision texture interpolator means that there's a maximum of 256 intermediate values possible between two adjacent pixels of a texture. 256 values are a lot for albedo textures for sure, but often in computer graphics textures encode not only surface properties, but they serve as LookUp Tables (LUT), heightfields (for terrain rendering), or who knows what. In those cases, you can find yourself easily lacking more resolution than 256 values between pixels. This article is about why this problem manifests and how it can be easily workarounded. In the image below you can see the difference between a regular GLSL's `texture()` or `texture2D()` call which triggers the hardware texture interpolation with its 256 intermediate values and that produces staircase artifacts versus the correct full floating point texture interpolation which produces the desired smooth results.



Terrain rendered with hardware texture interpolation (producing artifacts) vs manual texture interpolation (soft results) <https://www.shadertoy.com/view/MdBGzG>

The problem

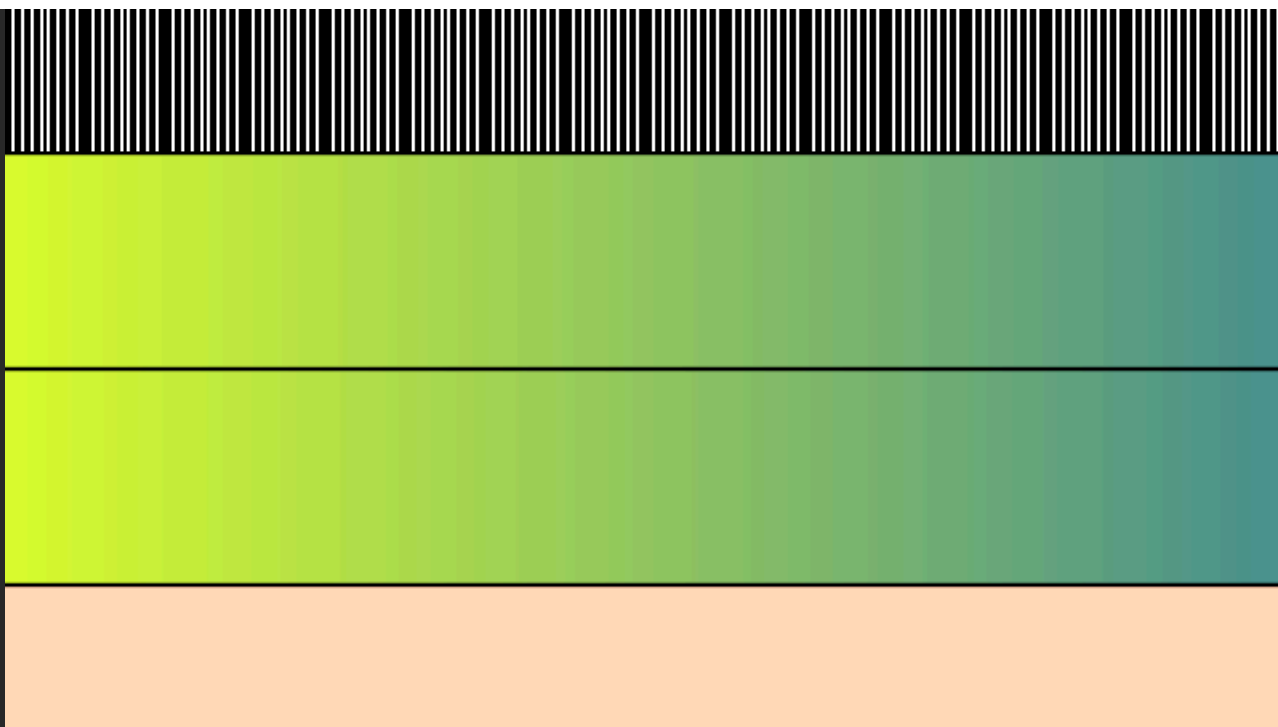
As said, the problem comes from the fact that the hardware is using only 8 bits for the fractional part of the texel interpolator. Once the vertex UVs have been converted into texel coordinates, the integer part of the coordinates determine the pixels to be taken into account in the interpolation, and the fractional part feed the interpolation formula. For easier visualization, imagine we were working with a 1024x1 pixels grayscale 1D texture, and that we were reading from it at pixel 53 and pixel 54, at a sampling rate of 500 steps (say we are zooming in it). Imagine pixel 53 contains the pixel value 10 and that the pixel 54 contains the pixel value 11. Then, if we computed the first 5 of the 500 values, we'd expect to get 10.000, 10.002, 10.004, 10.006, 10.008 (the sequence would go increasing all the way to 11.0, in steps of $0.002 = 1/500$). That's what the first column represents in the table below. However, because of the 8 bit precision interpolants, the results look very different, as you can see in the second column below:

Alpha	Expected	Real
0.000	$10 * (1.0 - 0.000) + 0.000 * 11 = \mathbf{10.000}$	$(10 * (256 - 0) + 0 * 11) \gg 8 = \mathbf{10.000}$

0.002	$10*(1.0-0.002)+0.002*11 = \mathbf{10.002}$	$(10*(256-1)+1*11)>>8 = \mathbf{10.004}$
0.004	$10*(1.0-0.004)+0.004*11 = \mathbf{10.004}$	$(10*(256-1)+1*11)>>8 = \mathbf{10.004}$
0.006	$10*(1.0-0.006)+0.006*11 = \mathbf{10.006}$	$(10*(256-2)+2*11)>>8 = \mathbf{10.008}$
0.006	$10*(1.0-0.008)+0.008*11 = \mathbf{10.008}$	$(10*(256-2)+2*11)>>8 = \mathbf{10.008}$

The third column in the table above displays what the hardware texture unit is really returning to us. As you can see, since 8 bits are used for the fixed point interpolation, the 0.002 step size cannot be represented. The smallest granularity of the interpolation is $1/256 = 0.0039$, which is bigger. So, the interpolant will repeat itself every few times before it jumps to the next representable value. That means, that the colors the texture unit returns to as are not continuous values between 10.0 and 11.0, but some sort of staircase.

The image below is a screenshot of this shader <https://www.shadertoy.com/view/Xsf3Dr> that you can use to see the effect in your own computer. The image shows in its top half the hardware texture unit in action, interpolating between yellow and blue. The color gradient looks pretty smooth to the eye. However, taking derivatives of the signal highlights the problem. If the values in the gradient were smooth indeed, then derivative of the gradient would be constant (the derivative of a line is a constant with the value of the slope of the line). However, rather than getting a constant color, the derivative (shown in the top part of the image) shows a series of black and white areas. The black areas are the pixels in which the interpolator didn't change values due to the reduced resolution of the 8 bit arithmetic, just as in the numerical table above. The white lines are the pixels at which the interpolant got assigned a new value.



Realtime demonstration of the problem: <https://www.shadertoy.com/view/Xsf3Dr>

The second half of the image shows the correct linear interpolation of the same two colors, which looks visually very similar to the hardware interpolator. The bottom part of the image shows a constant color, which is the expected constant derivative between the blue and yellow colors.

How to workaround it

The fix to the problem is easy - do not use the hardware interpolator, but do the bilinear filtering by hand. Or in other words, rather than doing:

```
// regular texture fetching
vec4 textureBad( sampler2D sam, vec2 uv )
{
    return texture( sam, uv );
}
```

do this instead:

```
// improved bilinear interpolated texture fetch
vec4 textureGood( sampler2D sam, vec2 uv )
{
    ivec2 ires = textureSize( sam, 0 );
    vec2 fres = vec2( ires );
}
```

```

vec2 st = uv*fres - 0.5;
vec2 i = floor( st );
vec2 w = fract( st );

vec4 a = texture( sam, (i+vec2(0.5,0.5))/fres );
vec4 b = texture( sam, (i+vec2(1.5,0.5))/fres );
vec4 c = texture( sam, (i+vec2(0.5,1.5))/fres );
vec4 d = texture( sam, (i+vec2(1.5,1.5))/fres );

return mix(mix(a, b, w.x), mix(c, d, w.x), w.y);
}

```

or if you in a modern GLSL dialect, and don't care about mipmapping, then you can also do

```

// improved bilinear interpolated texture fetch
vec4 textureGood( sampler2D sam, vec2 uv )
{
    ivec2 ires = textureSize( sam, 0 );
    vec2 fres = vec2( ires );

    vec2 st = (fract(uv)-0.5/fres)*fres;
    ivec2 i = ivec2( floor( st ) );
    vec2 w = fract( st );

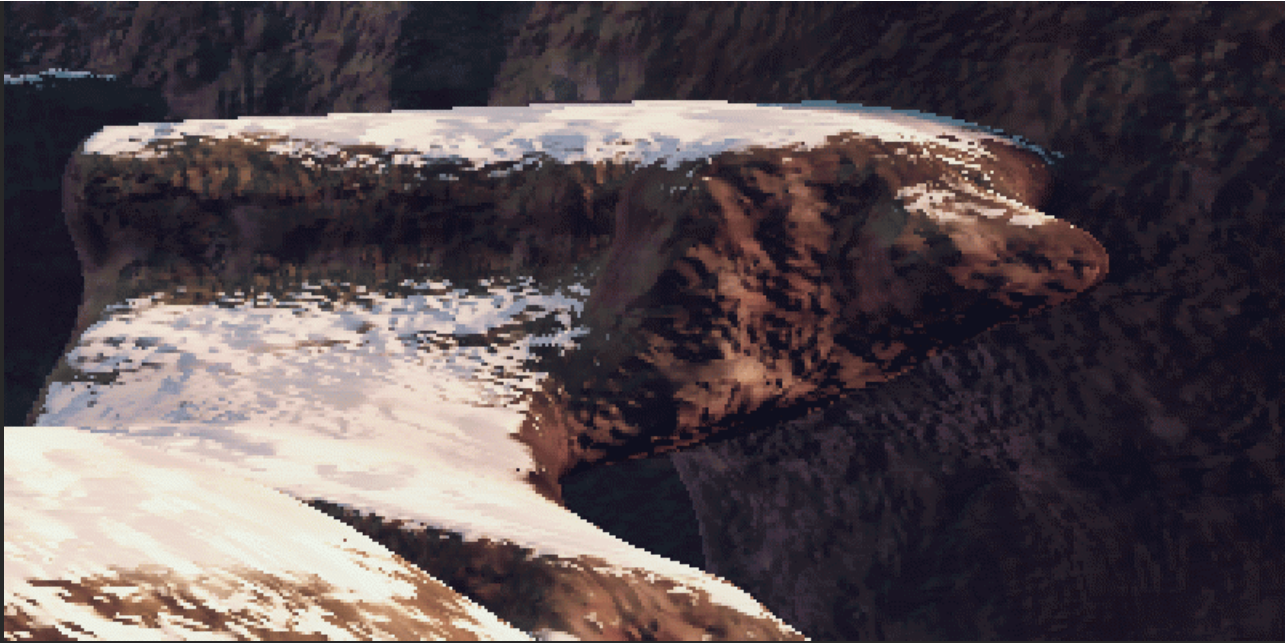
    vec4 a = texelFetch( sam, (i+ivec2(0,0)), 0 );
    vec4 b = texelFetch( sam, (i+ivec2(1,0)), 0 );
    vec4 c = texelFetch( sam, (i+ivec2(0,1)), 0 );
    vec4 d = texelFetch( sam, (i+ivec2(1,1)), 0 );

    return mix(mix(a, b, w.x), mix(c, d, w.x), w.y);
}

```

Of course the solution requires 4 times the amount of texture fetches, which might or might not be a problem depending on your application. Luckily these are cache friendly, so probably it is not *that* bad. You might want to use textureGather() for grey-scale textures to get the four pixel values instead in a single call.

Depending on the application, the visual difference between the default texture interpolation and the improved one can be massive, as seen in the first image of the article which is magnified below. This type of error is even more observable under animated cameras.



inigo quilez - learning computer graphics since 1994